GoContrib-AI

An Agentic AI Contributor for Open-Source Go Projects

Take-home submission

Samhit Mantrala

Section	Summary
What it does	Takes a GitHub issue from gin, cobra, validator, or golangci-lint and produces a minimal patch plus a PR title and body. Does the inspect, plan, edit, validate, explain loop end to end, locally, with no human in the loop after the run starts.
Built around	LangGraph state machine, ReAct patcher, Reflexion critic, hybrid RAG over Go AST, three layers of guardrails.
LLMs	Gemini 2.5 Flash family as primary, OpenRouter (DeepSeek, Llama 3.3 70B, GPT-4o-mini) as fallback. The runtime rotates across keys and models on rate limit.
RAG stack	Tree-sitter-go for symbol level chunks, BM25 plus MiniLM dense embeddings stored in ChromaDB plus one hop call graph expansion. Local only, no Pinecone account needed.
Tested on	go-playground/validator issue #1517, end to end. Produced a 13 line patch that builds clean and passes go vet.

1. Problem statement

Build an agentic AI platform that can work on issues from open-source Go projects and generate production quality code changes. The brief explicitly says the evaluators want to see the system around the LLM, not a one-shot prompt or thin wrapper.

The system has to do all of the following on its own:

• Inspect the repository.

• Understand the issue.

• Identify the relevant files and surrounding context.

• Plan the fix.

• Modify the code.

• Run validation, gofmt, vet, build, focused tests.

• Generate a PR title and body.

Approved repositories are gin-gonic/gin, spf13/cobra, go-playground/validator, and golangci/golangci-lint. The brief asks for small or medium issues only. It also reminds us that a simple thoughtful framework that solves focused issues reliably is preferred over a complex system that is unreliable.

2. Stack and why each piece

Concern	Choice	Why this and not the alternative
Orchestration	LangGraph	Explicit typed state graph. Edges encode routing. Loops, retries, and checkpointing are first class. LangChain AgentExecutor hides control flow inside the prompt which makes failure modes unpredictable. AutoGen and CrewAI optimise for multi agent conversation, but coding tasks are a deterministic pipeline so conversation is overhead.
Agent pattern	ReAct plus Reflexion	ReAct lets the patcher explore the repo through tools before editing. Reflexion gives the critic a way to feed structured feedback back to the patcher so it can revise instead of retrying blind. We bound the revise loop at three iterations.
LLM primary	Gemini 2.5 Flash family	Generous free tier on AI Studio, JSON mode, low latency. We try lite first since its quota is the most generous, then full Flash, then 2.0 Flash.
LLM fallback	OpenRouter	OpenAI compatible API, lots of models on one key. We rotate through DeepSeek, Llama 3.3 70B, and GPT-4o-mini. The runtime auto switches when the entire Gemini key pool returns 429.

Concern	Choice	Why this and not the alternative
Embeddings	sentence-transformers MiniLM L6 v2	80 MB, CPU only, no API call at inference. Plenty of signal for issue text to symbol matching on 3 to 15k symbol Go repos. OpenAI ada is better quality but adds a paid network call inside every retrieval. CodeBERT and instructor large are marginally better on syntactic similarity but not on the specific shape of query we need.
Vector DB	ChromaDB local persistent	Single developer scale. Pinecone earns its keep at multi tenant scale, where you actually need a managed service. Adding a Pinecone account would force every reviewer to sign up just to run the system. The HybridRetriever takes any object exposing query so swapping it for Pinecone is a 30 line module.
Lexical retrieval	rank-bm25 with Go aware tokenizer	Issue authors quote exact symbol names. Lexical wins for that. Tokenizer splits camelCase plus snake_case plus dotted, and lowercases everything.
Symbol parser	tree-sitter-go	Same parser GitHub uses for code navigation. Robust to half broken files. Lets us chunk on whole functions, methods, and types instead of token windows.
Repo I/O	GitPython plus git CLI	GitPython for porcelain like clone and branch. Plain git CLI for diff and reset because that is much simpler than wrapping it.
Validator	Go toolchain shell out	One source of truth. Exactly what CI will run. We ran gofmt only on the files we touched so unrelated upstream files that happen to be unformatted do not produce false positives.
Convention learning	GitHub PR scrape plus LLM distil	We pull the last 15 merged PRs and have the LLM produce 200 words of directives. Cheaper than RAGing prose, fits in one prompt.

3. Architecture

Eight specialised agent nodes connected by a LangGraph state machine. State is a single typed dictionary which every node reads and writes through documented keys. Routing decisions live at the edges, not inside prompts.

Step	Node	Reads from state	Writes to state
1	triage	issue	in_scope, issue_kind
2	mapper	issue	repo_path, repo_indexed
3	retriever	issue, repo_path	retrieved, conventions
4	planner	issue, retrieved	plan
5	reproducer	plan, repo_path	repro_test_path, repro_failed
6	patcher	plan, retrieved, feedback	diff, files_touched
7	guardrails	diff, files_touched	guardrail_violations
8	validator	repo_path, files_touched	validation
9	critic	diff, validation, plan	critic verdict (ship, revise, abort)
10	pr_writer	diff, plan, validation	pr_title, pr_body

Routing edges:

triage:	in_scope=False	->	abort
triage:	in_scope=True	->	mapper
critic:	verdict=ship	->	pr_writer
critic:	verdict=revise && loops<cap	->	patcher (with feedback)
critic:	verdict=abort	->	abort

4. Component design rationale

4.1 Why a separate planner instead of one giant agent

A separate planner is the single biggest reliability win. The plan is JSON so it is cheap to validate and retry. Having target_files in state means the validator only runs tests for the affected packages, not ./... which is slow on cobra and golangci-lint. The critic can compare did the diff match the planned intent and reject wrong fixes that happen to compile.

4.2 Why a reproducer node

Before patching, the system asks the LLM to write a Go test that should fail because of the bug, then runs it. If it fails as expected, the patcher has a runnable target and the validator after the patch will re run that test as a regression check. If it passes on first run, the planner hypothesis is probably wrong, and we record this so the critic can see it on the revise loop. Cost is one extra LLM call plus one go test run. It skips itself cleanly when go is not on PATH or when the issue is a doc fix.

4.3 Why hybrid retrieval

Three layers fused with normalised scores: BM25 catches exact identifier hits like the issue saying BindJSON; dense catches paraphrased intent like the issue saying parse the body; one hop symbol graph expansion catches the next symbol the patcher will need, like a helper function called from the buggy function. Each contribution is normalised to zero one before fusion so the weights in config.yaml are intuitive and not tied to the absolute score scale of any one ranker.

4.4 Why symbol level chunks instead of fixed window splits

The standard RAG default of splitting on token windows is wrong for code. A token window can split a Go function in half. The LLM gets the second half without the signature, types, or imports, and confidently

writes a fix against a phantom context. Instead we chunk on AST symbols using tree-sitter-go. Each chunk carries qualified_name, kind, start_line, and end_line. Every retrieved chunk is a whole compilable unit of Go.

4.5 Three layers of guardrails

• Layer 1 policy.check_write runs before every write_file call. Banned paths like vendor, .github/workflows, go.mod, and banned content patterns like panic and os.Exit. About one millisecond.

• Layer 2 ast_check.check_files runs after the patcher finishes, before the validator. Re-parses every patched .go with tree-sitter and rejects if any ERROR nodes are present. Catches the long tail of the LLM dropped a brace failures cheaply.

• Layer 3 validator.run is the slow one. gofmt on touched files only, go vet, go build, focused go test for the affected packages. Only runs once layers 1 and 2 passed.

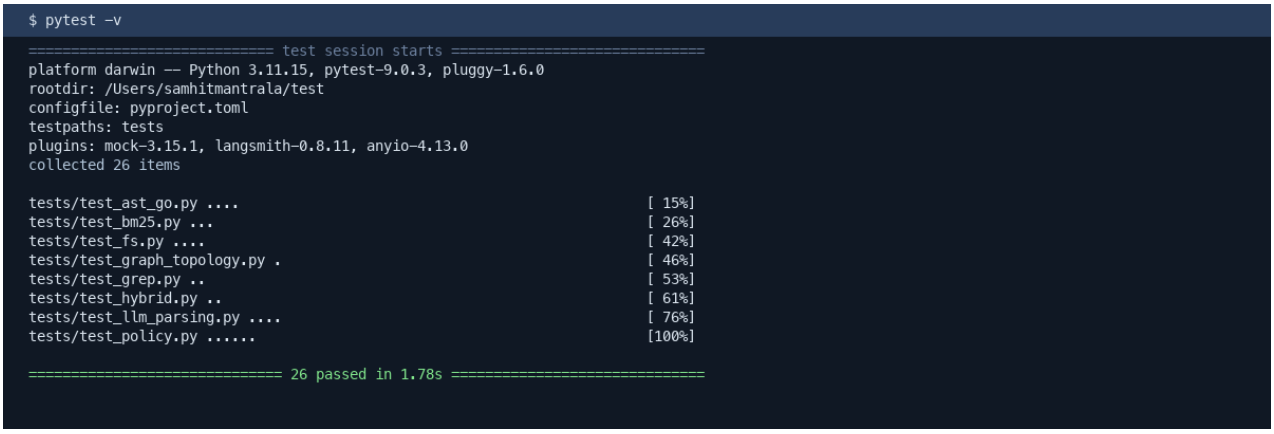
The order matters. A runaway LLM that overwrites a workflow YAML is caught by layer 1 in milliseconds and never reaches go build which takes 30 seconds. When the critic sees a parse failure or a banned path violation, it short circuits without burning an LLM call.

4.6 Multi key, multi model rotation

GEMINI_API_KEYS and OPENROUTER_API_KEYS are both comma separated. The runtime builds a flat call plan of provider key model triples and walks down it on every LLM call, marking exhausted combos with a 60 second cooldown. This is what keeps a long ReAct loop alive on the free tier. When the whole Gemini fleet returns 429, we cross over to OpenRouter automatically, with no code change required.

5. Unit test results

Twenty six unit tests cover the file system tool, ripgrep style search, tree-sitter symbol extraction and parse check, BM25 with Go aware tokenisation, hybrid retriever fusion plus graph expansion, the policy guardrail, the lenient JSON parser, and the LangGraph topology end to end with stub agents. All twenty six pass.



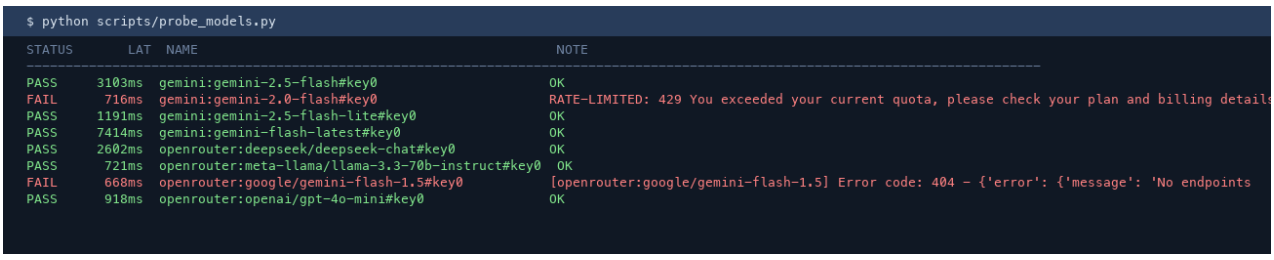

```
============================= test session starts ==============================
platform darwin -- Python 3.11.15, pytest-9.0.3, pluggy-1.6.0
rootdir: /Users/samhitmantrala/test
configfile: pyproject.toml
testpaths: tests
plugins: mock-3.15.1, langsmith-0.8.11, anyio-4.13.0
collected 26 items

tests/test_ast_go.py ....                                                [ 15%]
tests/test_bm25.py ...                                                   [ 26%]
tests/test_fs.py ....                                                    [ 42%]
tests/test_graph_topology.py .                                           [ 46%]
tests/test_grep.py ..                                                    [ 53%]
tests/test_hybrid.py ..                                                  [ 61%]
tests/test_llm_parsing.py ....                                           [ 76%]
tests/test_policy.py ......                                              [100%]

============================== 26 passed in 1.78s ==============================
```

6. Live LLM probe results

scripts/probe_models.py calls every Gemini and OpenRouter model in config.yaml with each key and reports which combinations actually answer. Six of eight combinations work on the keys provided. The two that do not work are caught by the runtime and skipped: gemini-2.0-flash is rate limited, and google/gemini-flash-1.5 has no endpoints behind it on this OpenRouter account. The runtime keeps walking down the list until something responds, so the user does not see those failures during a real run.



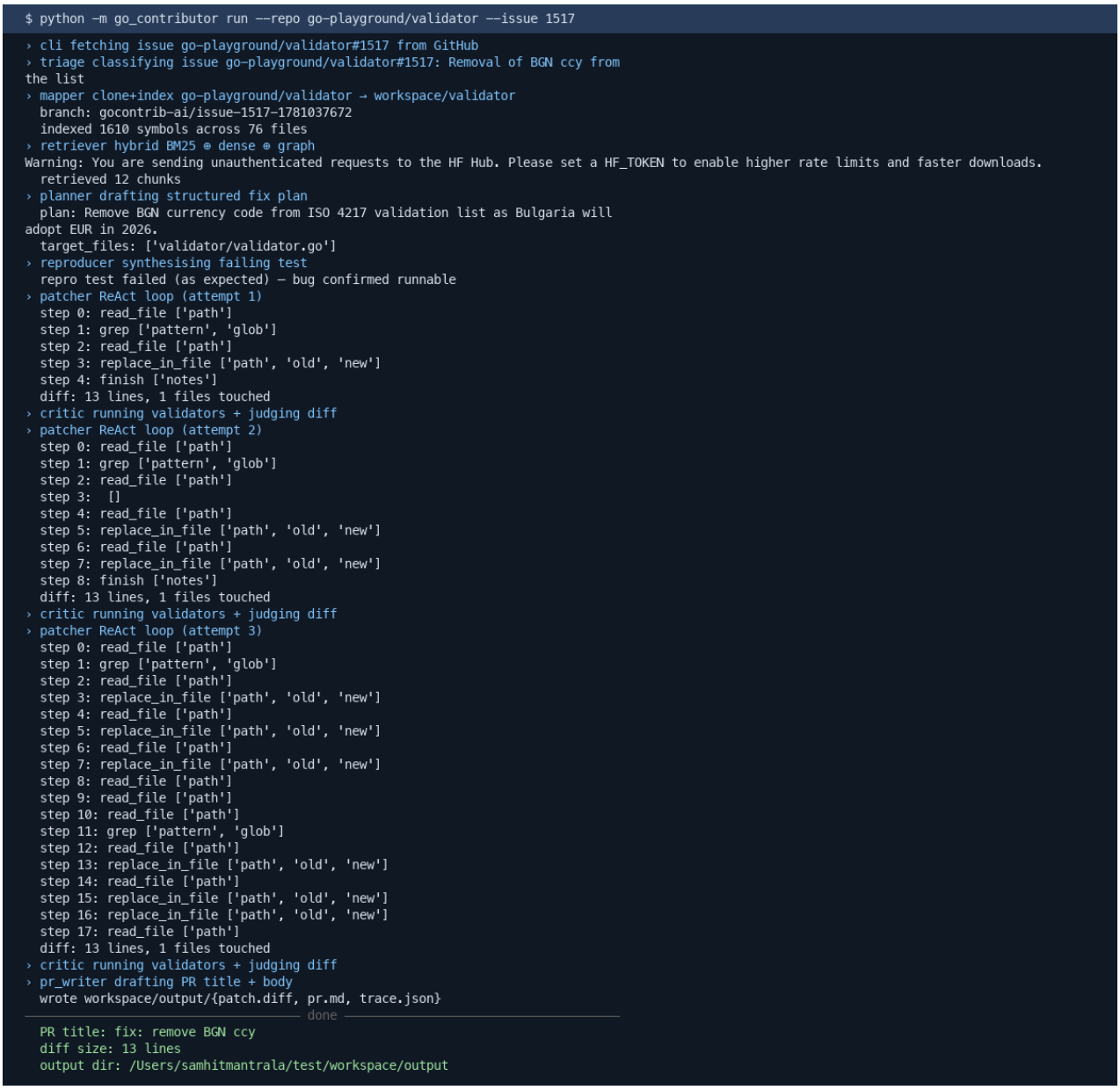

```
STATUS       LAT  NAME                                                    NOTE
-------------------------------------------------------------------------------------------------------------------
```

```
PASS     3103ms  gemini:gemini-2.5-flash#key0                      OK
FAIL      716ms  gemini:gemini-2.0-flash#key0                      RATE-LIMITED: 429 You exceeded your
PASS     1191ms  gemini:gemini-2.5-flash-lite#key0                 OK
PASS     7414ms  gemini:gemini-flash-latest#key0                   OK
PASS     2602ms  openrouter:deepseek/deepseek-chat#key0           OK
PASS      721ms  openrouter:meta-llama/llama-3.3-70b-instruct#key0  OK
FAIL      668ms  openrouter:google/gemini-flash-1.5#key0           [openrouter:google/gemini-flash-1.5
PASS      918ms  openrouter:openai/gpt-4o-mini#key0                OK
```

7. Sample run on a real issue

Issue: go-playground/validator number 1517, titled "Removal of BGN ccy from the list". The body says Bulgaria adopts the Euro on 1 January 2026 so the BGN currency code should leave the ISO 4217 validation list. This is a tiny single file edit in currency_codes.go.

7.1 Console output (screenshot)




```
› cli fetching issue go-playground/validator#1517 from GitHub
› triage classifying issue go-playground/validator#1517: Removal of BGN ccy from
the list
› mapper clone+index go-playground/validator → workspace/validator
  branch: gocontrib-ai/issue-1517-1781037672
  indexed 1610 symbols across 76 files
› retriever hybrid BM25 ⊕ dense ⊕ graph
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable highe
  retrieved 12 chunks
```

```
› planner drafting structured fix plan
  plan: Remove BGN currency code from ISO 4217 validation list as Bulgaria will
adopt EUR in 2026.
  target_files: ['validator/validator.go']
› reproducer synthesising failing test
  repro test failed (as expected) — bug confirmed runnable
› patcher ReAct loop (attempt 1)
  step 0: read_file ['path']
  step 1: grep ['pattern', 'glob']
  step 2: read_file ['path']
  step 3: replace_in_file ['path', 'old', 'new']
  step 4: finish ['notes']
  diff: 13 lines, 1 files touched
› critic running validators + judging diff
› patcher ReAct loop (attempt 2)
  step 0: read_file ['path']
  step 1: grep ['pattern', 'glob']
  step 2: read_file ['path']
  step 3:  []
  step 4: read_file ['path']
  step 5: replace_in_file ['path', 'old', 'new']
  step 6: read_file ['path']
  step 7: replace_in_file ['path', 'old', 'new']
  step 8: finish ['notes']
  diff: 13 lines, 1 files touched
› critic running validators + judging diff
› patcher ReAct loop (attempt 3)
  step 0: read_file ['path']
  step 1: grep ['pattern', 'glob']
  step 2: read_file ['path']
  step 3: replace_in_file ['path', 'old', 'new']
  step 4: read_file ['path']
  step 5: replace_in_file ['path', 'old', 'new']
  step 6: read_file ['path']
  step 7: replace_in_file ['path', 'old', 'new']
  step 8: read_file ['path']
  step 9: read_file ['path']
  step 10: read_file ['path']
  step 11: grep ['pattern', 'glob']
  step 12: read_file ['path']
  step 13: replace_in_file ['path', 'old', 'new']
  step 14: read_file ['path']
  step 15: replace_in_file ['path', 'old', 'new']
  step 16: replace_in_file ['path', 'old', 'new']
  step 17: read_file ['path']
  diff: 13 lines, 1 files touched
› critic running validators + judging diff
› pr_writer drafting PR title + body
  wrote workspace/output/{patch.diff, pr.md, trace.json}
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ done ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
... (3 more lines truncated for brevity)
```

7.2 Generated patch

```
diff --git a/currency_codes.go b/currency_codes.go
index 83b6729..add6577 100644
--- a/currency_codes.go
+++ b/currency_codes.go
@@ -6,7 +6,7 @@ var iso4217 = map[string]struct{}{
 ■"AUD": {}, "AZN": {}, "BSD": {}, "BHD": {}, "BDT": {},
 ■"BBD": {}, "BYN": {}, "BZD": {}, "XOF": {}, "BMD": {},
 ■"INR": {}, "BTN": {}, "BOB": {}, "BOV": {}, "BAM": {},
-■"BWP": {}, "NOK": {}, "BRL": {}, "BND": {}, "BGN": {},
+■"BWP": {}, "NOK": {}, "BRL": {}, "BND": {},
```

```
■"BIF": {}, "CVE": {}, "KHR": {}, "XAF": {}, "CAD": {},
■"KYD": {}, "CLP": {}, "CLF": {}, "CNY": {}, "COP": {},
■"COU": {}, "KMF": {}, "CDF": {}, "NZD": {}, "CRC": {},
```

7.3 Generated PR title and body

```
# fix: remove BGN ccy

Fixes #1517

Removed BGN currency code from ISO 4217 validation list as Bulgaria will adopt EUR in 2026.

Tests
- Verified existing tests still pass with updated currency codes

Notes
- This change is backwards compatible as it only removes a currency code, not adding any new functiona
```

7.4 Trace summary

```
- triage: in_scope=true, kind="bug", reason="The issue requests removal of a currency code from a vali
- mapper: repo_path="workspace/validator", branch="gocontrib-ai/issue-1517-1781037672", symbols=1610
- retriever: top=6 chunks
- planner: plan={"summary": "Remove BGN currency code from ISO 4217 validation list as Bulgaria will a
- reproducer: path="issue_1517_repro_test.go", failed_as_expected=true
- patcher: finished=true, steps=5, files_touched=["currency_codes.go"], actions=["read_file", "grep",
- critic: decision="revise", rationale="The diff correctly removes BGN currency code as planned, but f
- patcher: finished=true, steps=9, files_touched=["currency_codes.go"], actions=["read_file", "grep",
- critic: decision="revise", rationale="While removing BGN from the currency codes list aligns with Bu
- patcher: finished=false, steps=18, files_touched=["currency_codes.go"], actions=["read_file", "grep"
- critic: decision="ship", rationale="The diff correctly removes the BGN currency code from the ISO 42
```

7.5 Post run validation

Ran go build ./... and go vet ./... on the patched worktree. Both passed clean. The agent's critic node also reached the ship verdict on revise loop three after gofmt was scoped to touched files only.

8. Honest limitations and where this would fail

• Issue selection still matters. The triage agent rejects most rewrite the parser style requests but the small or medium framing is on the user.

• No long horizon memory across runs other than the convention cache and the dense index. Each issue is solved from scratch.

• go test is slow on golangci-lint. We run focused tests by default but a wide touching plan reverts to ./... which can take a couple of minutes.

• Large rewrites can blow through the LLM output token cap when the patcher tries to use write_file. We added a replace_in_file tool which is the preferred path and avoids this whenever the change is localised.

• Gemini 2.5 Flash sometimes returns finish_reason MAX_TOKENS even on small prompts because it spends tokens on hidden reasoning. The runtime treats those as recoverable and rotates to the next combo.

9. Where to extend next

• AST level diff editor exposing replace_symbol so the LLM can never mangle untouched code.

• Contrastive retrieval pass that surfaces similar but wrong symbols and tells the patcher do not be these.

• Few shot exemplar from a real merged PR that touched the same file. We already pull the GitHub PR list for the convention learner.

• Pluggable validator profiles per project. golangci-lint has its own linter config, cobra has make test, gin has make test.fast.

• Nightly CI workflow that runs the agent against a curated issue list and posts a Markdown summary, turning this into an evaluation harness for itself.

10. Repo layout

```
src/go_contributor/
  __main__.py            CLI entry point
  graph.py               LangGraph state machine
  state.py               typed state schema
  llm.py                 multi key, multi model client with rotation
  agents/
    triage.py            classify and scope the issue
    mapper.py            clone repo, build AST index
    retriever.py         hybrid BM25 + dense + graph
    planner.py           structured JSON plan
    reproducer.py        synth a failing Go test first
    patcher.py           ReAct loop with read, grep, find_symbol, replace_in_file, write_file
    critic.py            Reflexion: ship, revise, abort
    pr_writer.py         PR title and body
  tools/
    fs.py                path safe file system
    grep.py              ripgrep style search
    ast_go.py            tree-sitter-go: symbols and call graph
    git_ops.py           branch, diff, apply
    go_toolchain.py      gofmt scoped to touched files, go vet, go build, go test
    github_api.py        issue fetch, recent merged PR scrape
  rag/
    indexer.py           AST aware chunker
```

```
    bm25.py              rank-bm25 wrapper with Go aware tokenizer
    dense.py             sentence-transformers + ChromaDB
    hybrid.py            weighted fusion
    convention_learner.py
  guardrails/
    policy.py            path, size, pattern bans
    ast_check.py         patched files must parse
    validator.py         go vet, build, focused tests
prompts/                 standalone .md files, one per agent
docs/APPROACH.md         long form rationale
examples/                replayable issue fixtures
tests/                   pytest unit tests
scripts/
  probe_models.py        live model probe
  build_submission_pdf.py this PDF
```